

E-STORE

Guide de démarrage & Soutenance

Comment lancer le projet et présenter la démo

Module	Full-Stack
Encadrant	Pr. Omar Zahour
Établissement	FSBM — Université Hassan II
Année	2025-2026
Auteurs	Akram Belmoussa & Nouhaila Ben Soumane
Document	7/7 d'une série explicative

1. Pré-requis

Logiciel	Version	Vérification
JDK	17 ou +	java -version
Maven	3.9 ou +	mvn -version
Node.js	20 ou +	node --version
npm	10 ou +	npm --version

2. Démarrage en 3 étapes

Étape 1 — (Optionnel) Lancer les BDD via Docker

```
cd C:\Users\belmo\studies\Full-stack\E-stor
docker compose up -d
```

Si Docker n'est pas installé, **passer cette étape** — l'application démarrera avec H2 in-memory (profil dev par défaut).

Étape 2 — Lancer le backend Spring Boot

```
cd estore-backend
mvn spring-boot:run
```

```
# Sortie attendue :
# Tomcat started on port 8080
# DataSeeder : 2 utilisateurs créés
# DataSeeder : 3 catégories + 12 produits créés
# Started EstoreApplication in 6.5 seconds
```

Étape 3 — Lancer le frontend Angular

```
cd estore-frontend
npm install      # première fois seulement
npm start
```

```
# Ouvre automatiquement http://localhost:4200
```

3. Comptes de test

Email	Mot de passe	Rôle	Permissions
admin@estore.ma	Admin@123	ADMIN	Catalogue, panier, commandes, CRUD produits/catégories
user@estore.ma	User@123	USER	Catalogue, panier, commandes, avis

4. Scénario de démonstration recommandé

Voici un parcours en 8 étapes qui couvre toutes les fonctionnalités du projet — durée totale ~5 minutes.

1. Ouvrir <http://localhost:4200> — montrer le catalogue avec 12 produits, la pagination, le filtre par catégorie.
2. Cliquer sur un produit → fiche détaillée avec image, prix, stock, et liste des avis.
3. Tenter d'ajouter au panier sans être connecté → message "Veuillez vous connecter" + redirection /login.
4. Se connecter avec `user@estore.ma` / `User@123` → header met à jour le nom et le badge panier.
5. Ajouter 2 produits différents au panier → badge passe à 2, toast vert de confirmation.
6. Aller dans `/cart` → modifier les quantités, voir le total recalculé.
7. Cliquer "Valider la commande" → redirection `/orders`, voir la commande #1 confirmée.
8. Retourner sur un produit, déposer un avis 5 étoiles → l'avis apparaît immédiatement dans la liste.

5. Points forts à mettre en avant

Architecture. 3 couches × 5 domaines, structure cohérente et scalable.

Sécurité. JWT + BCrypt + Spring Security 6, @PreAuthorize sur les endpoints ADMIN.

Persistance hybride. JPA pour la cohérence transactionnelle, MongoDB pour les avis volumineux.

Robustesse. Tout le checkout est transactionnel (@Transactional) — atomicité garantie.

Tests. 9 tests unitaires JUnit 5 + Mockito, BUILD SUCCESS.

DevOps. Profils dev/prod, Docker Compose multi-services, fallback H2 sans installation.

UI/UX. Angular 17 standalone + signals, Bootstrap 5, toasts, lazy loading, responsive.

Documentation. README complet + 7 PDF explicatifs (ce document).

6. Questions courantes du jury et réponses

Q : Pourquoi avoir choisi MongoDB pour les avis et pas SQL ?

Les avis sont des données semi-structurées, peu contraintes relationnellement, lues très fréquemment et écrites en flux tendu. MongoDB offre de meilleures performances en lecture/écriture pour ce type de données et permet d'illustrer la coexistence de deux paradigmes de persistance dans une même application.

Q : Comment garantisseriez-vous la cohérence du panier en cas de stock insuffisant ?

L'opération checkout est annotée @Transactional. Toutes les vérifications de stock sont effectuées AVANT toute modification. Si une vérification échoue, une BusinessException est levée → le rollback automatique annule toutes les opérations déjà effectuées dans la transaction.

Q : Pourquoi H2 en plus de MySQL ?

H2 est utilisée pour le profil dev — démarrage instantané sans installation, idéale pour les démos et le développement local. MySQL est utilisée pour la production. Le code est strictement identique grâce à JPA.

Q : Comment le JWT est-il sécurisé ?

Signé en HMAC-SHA256 avec une clé secrète configurée hors du code (application.properties). Expiration 24h. Vérifié à chaque requête par JwtAuthenticationFilter avant exécution du controller.

Q : Pourquoi Angular standalone et pas un NgModule classique ?

C'est le standard depuis Angular 17. Plus concis, plus performant (lazy loading natif), pas de boilerplate. Les composants déclarent leurs dépendances directement.

7. Liens utiles pendant la démo

Adresse	Contenu
http://localhost:4200	Frontend Angular
http://localhost:8080/api/products	API REST (lecture publique)
http://localhost:8080/h2-console	Console H2 (profil dev)
http://localhost:8081	phpMyAdmin (si Docker)
http://localhost:8082	mongo-express (si Docker)